FIAP · Análise e Desenvolvimento de Sistemas · Navegando pelo mundo DevOps

Projeto - Cidades ESG Inteligentes

CoMed ESG — CI/CD, containerização e deploy

Documentação técnica · Spring Boot + MongoDB · GitHub Actions · Docker · Render · Atlas

Integrantes

Nome	RM	Turma
Lucas Gabriel Neres Sant'Ana	565612	2TDSOC
Matheus Ferreira da Costa	564003	2TDSOC

Links do projeto

Recurso	Endereço
Repositório	github.com/Matheus-Costa2004/comed-esg
Pipeline (Actions)	github.com/Matheus-Costa2004/comed-esg/actions
Staging	comed-esg-staging.onrender.com
Produção	comed-esg-prod.onrender.com
Imagem Docker	ghcr.io/matheus-costa2004/comed-esg

Resumo

O CoMed é um copiloto de IA clínica para médicos e pacientes. Nesta entrega, a API foi reconstruída em **Java 21 + Spring Boot 3.5** sobre o modelo de dados **MongoDB** organizado pelos pilares ESG (pacientes, médicos, consultas, alertas de IA e auditoria LGPD) e recebeu um ciclo DevOps completo: testes automatizados, imagem Docker multi-stage, orquestração com Docker Compose e um pipeline **GitHub Actions** que publica a imagem no GHCR e faz o deploy automatizado em **staging** e, após aprovação manual, em **produção**, ambos no Render com banco no MongoDB Atlas.

1. Visão geral da arquitetura

Um único *push* na branch `main` percorre todo o fluxo abaixo. A mesma imagem (identificada pelo SHA do commit) é promovida de staging para produção, então o que foi validado em staging é exatamente o que vai para produção.

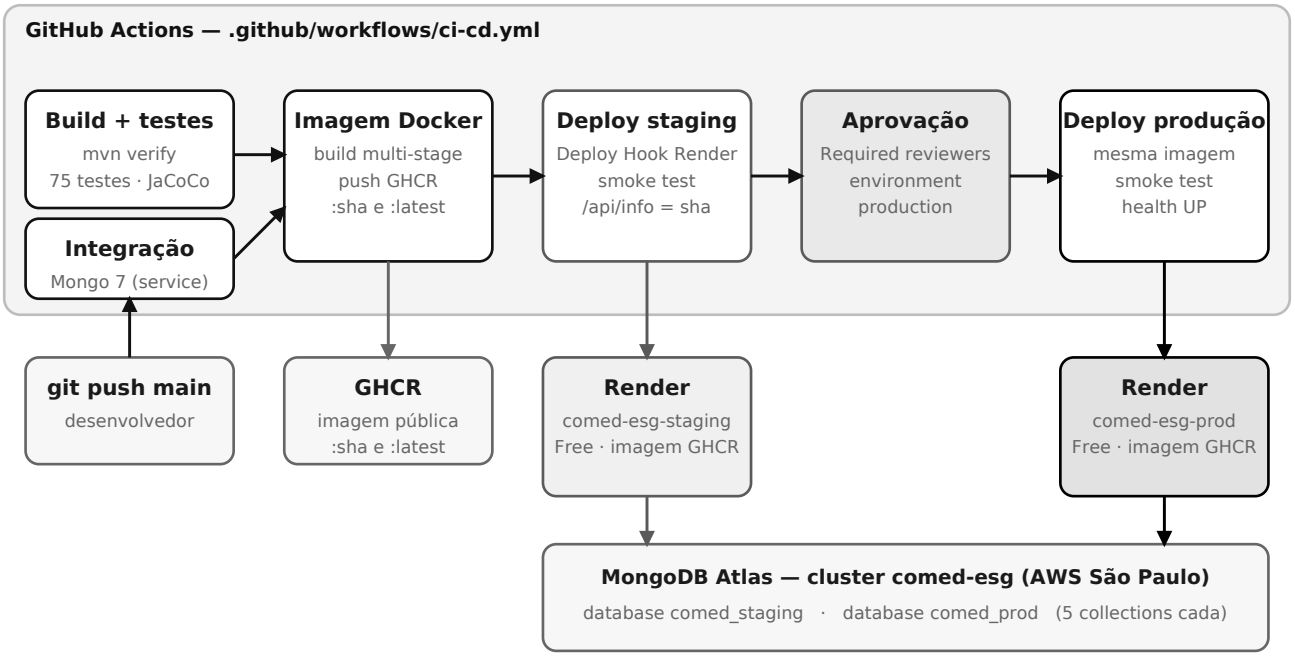


Figura 1 — Fluxo do pipeline: build e testes, imagem no GHCR, deploy em staging, aprovação e deploy em produção.

Ambientes

Ambiente	Onde roda	Banco	Como é implantado
Local	Docker Compose (máquina ou GitHub Codespace)	Container mongo:7 com volume	docker compose up -d --build
Staging	Render Web Service `comed-esg-staging`	Atlas · comed_staging	Automático a cada push na main
Produção	Render Web Service `comed-esg-prod`	Atlas · comed_prod	Após aprovação manual no GitHub

A página inicial da aplicação (`/`) é um painel de status que consome `/api/info` e `/api/esg/indicadores`: mostra o selo do ambiente (cinza = local, âmbar = staging, verde = produção), o SHA do commit em execução, o health e os indicadores ambientais, sociais e de governança. Ela é a evidência visual de cada ambiente.

2. Pipeline CI/CD

Ferramenta: GitHub Actions, com um workflow em `.github/workflows/ci-cd.yml`. **Gatilhos:** push em `main` e `develop`, pull request para `main` e execução manual (`workflow_dispatch`). Em PRs e na develop rodam só os testes; imagem e deploys acontecem apenas em push na main.

Etapas

Job	O que faz	Depende de
Build e testes unitários	Java 21 (Temurin) com cache Maven; `./mvnw -B verify` compila, roda 75 testes (Mockito + WebMvcTest) e gera a cobertura JaCoCo. Publica o resumo no Summary e os relatórios como artifact.	—
Testes de integração (MongoDB)	Sobe um `mongo:7` como service container e roda `mvn verify -Pintegration` (testes *IT com @SpringBootTest contra banco real).	—
Imagem Docker → GHCR	Build multi-stage com `--build-arg APP_VERSION=<sha>`, login no GHCR com o GITHUB_TOKEN e push das tags `:sha` e `:latest`.	build + integração
Deploy em staging	Environment `staging`. Chama o Deploy Hook do Render com `imgURL=…:<sha>` e faz smoke test: consulta `/api/info` a cada 15 s (até 12 min) até `versao == sha` e confere `/actuator/health = UP`.	imagem
Deploy em produção	Environment `production` com **Required reviewers**: o job só inicia após aprovação de um integrante. Mesmo script de deploy e smoke test, com a mesma imagem.	staging

Lógica e decisões

- **Promoção da mesma imagem:** a imagem é construída uma única vez; staging e produção recebem a mesma tag de SHA. O smoke test só aprova o deploy quando o ambiente responde com aquele SHA, o que prova que a versão nova está no ar.
- **Staging contínuo, produção com gate:** staging não tem regra de proteção (deploy automático); produção exige revisão de Matheus-Costa2004 ou devlucassantana — um controle de governança coerente com o pilar G do projeto.
- **Segredos fora do código:** os Deploy Hooks ficam como *environment secrets* (`RENDER_DEPLOY_HOOK_STAGING/PROD`) e as URLs como *variables* (`STAGING_URL/PROD_URL`). A `MONGODB_URI` com a senha do Atlas existe só no Render.
- **Concorrência:** um pipeline por branch e `cancel-in-progress: false`, para nunca interromper um deploy no meio.
- **Falha clara:** se um secret estiver ausente, o job falha com uma mensagem explicando o que cadastrar.

Execução registrada

Job	Duração	Resultado
Build e testes unitários	50 s	75/75 testes, cobertura de instruções 52,5%
Testes de integração (MongoDB)	1 min 13 s	Sucesso
Imagem Docker → GHCR	1 min 1 s	ghcr.io/matheus-costa2004/comed-esg:d0944c0…

Job	Duração	Resultado
Deploy em staging	1 min 53 s	comed-esg-staging.onrender.com · versão d0944c0
Deploy em produção	1 min 51 s	comed-esg-prod.onrender.com · versão d0944c0

Duração total da execução: 11 min 19 s, incluindo o tempo de espera pela aprovação de produção.

3. Docker: arquitetura, comandos e imagem

Dockerfile multi-stage

- **Estágio build** (`maven:3.9-eclipse-temurin-21`): copia primeiro só o `pom.xml` e roda `dependency:go-offline`, criando uma camada de dependências reaproveitada enquanto o pom não muda. Depois compila o JAR e extrai as camadas do Spring Boot (`jarmode=tools extract --layers`).
- **Estágio runtime** (`eclipse-temurin:21-jre-alpine`): só a JRE, sem Maven nem código-fonte. As camadas são copiadas da que menos muda (dependências) para a que mais muda (aplicação), acelerando rebuilds e pulls.
- **Segurança:** roda com o usuário sem privilégios `comed`; o Java é o PID 1 (exec) e recebe o SIGTERM para shutdown gracioso.
- **Configuração por ambiente:** `ARG/ENV APP_VERSION` (SHA do commit), `PORT` (o Render injeta a sua) e `JAVA_OPTS=-XX:MaxRAMPercentage=75` para caber nos 512 MB do plano Free.
- **HEALTHCHECK** em `/actuator/health` e `EXPOSE 8080`.

Orquestração com Docker Compose

Serviço	Imagem	Destaques
mongo	mongo:7	Usuário/senha root do .env; volume `comed-mongo-data`; healthcheck com mongosh ping; porta 27017 exposta só em 127.0.0.1.
api	build local → comed-esg:local	`depends_on: service_healthy`; MONGODB_URI montada a partir do .env; volume `comed-api-logs` em /app/logs; healthcheck próprio; porta configurável (API_PORT).
mongo-express	mongo-express:1.0.2	Opcional, no profile `tools`, com basic auth, em 127.0.0.1:8081.

Os três serviços compartilham a rede bridge nomeada `comed-net`; os volumes nomeados preservam dados e logs entre reinícios; todas as credenciais vêm do `.env` (modelo versionado em `.env.example`; o `.env` real fica fora do git).

Comandos usados

```
# ambiente local (máquina com Docker ou GitHub Codespace)
cp .env.example .env
docker compose up -d --build
docker compose ps
docker compose logs api --tail 60
docker compose --profile tools up -d        # inclui o mongo-express

# pipeline (job 'Imagem Docker → GHCR')
docker build --build-arg APP_VERSION=$GITHUB_SHA \
  -t ghcr.io/matheus-costa2004/comed-esg:$GITHUB_SHA \
  -t ghcr.io/matheus-costa2004/comed-esg:latest .
docker push ghcr.io/matheus-costa2004/comed-esg:$GITHUB_SHA
docker push ghcr.io/matheus-costa2004/comed-esg:latest

# usar a imagem publicada
docker pull ghcr.io/matheus-costa2004/comed-esg:latest
```

Imagem criada

`ghcr.io/matheus-costa2004/comed-esg`, pública no GitHub Container Registry, com as tags `latest` e `d0944c09dec02dbe43a8b8e7f5582868ee07dbe9` (SHA do commit). É essa imagem que os serviços de staging e produção do Render executam.

4. Evidências do pipeline

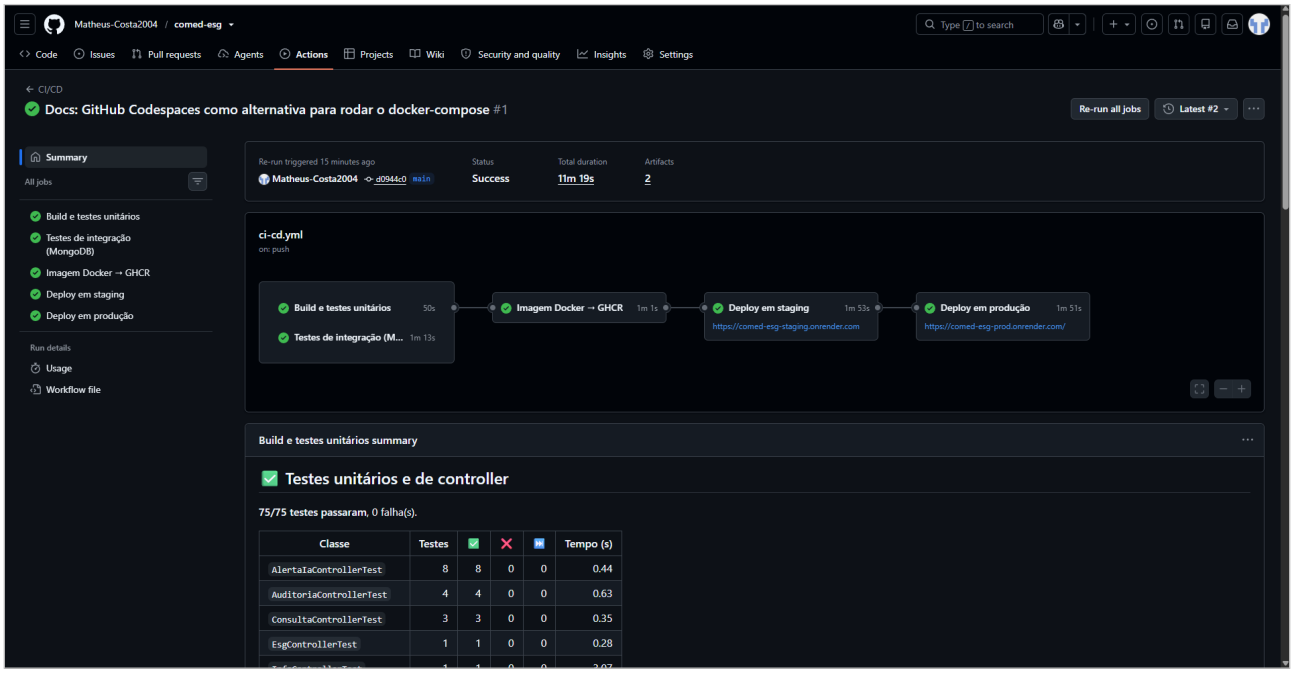


Figura 2 — Execução completa com status Success: os cinco jobs verdes e as URLs de staging e produção nos jobs de deploy.

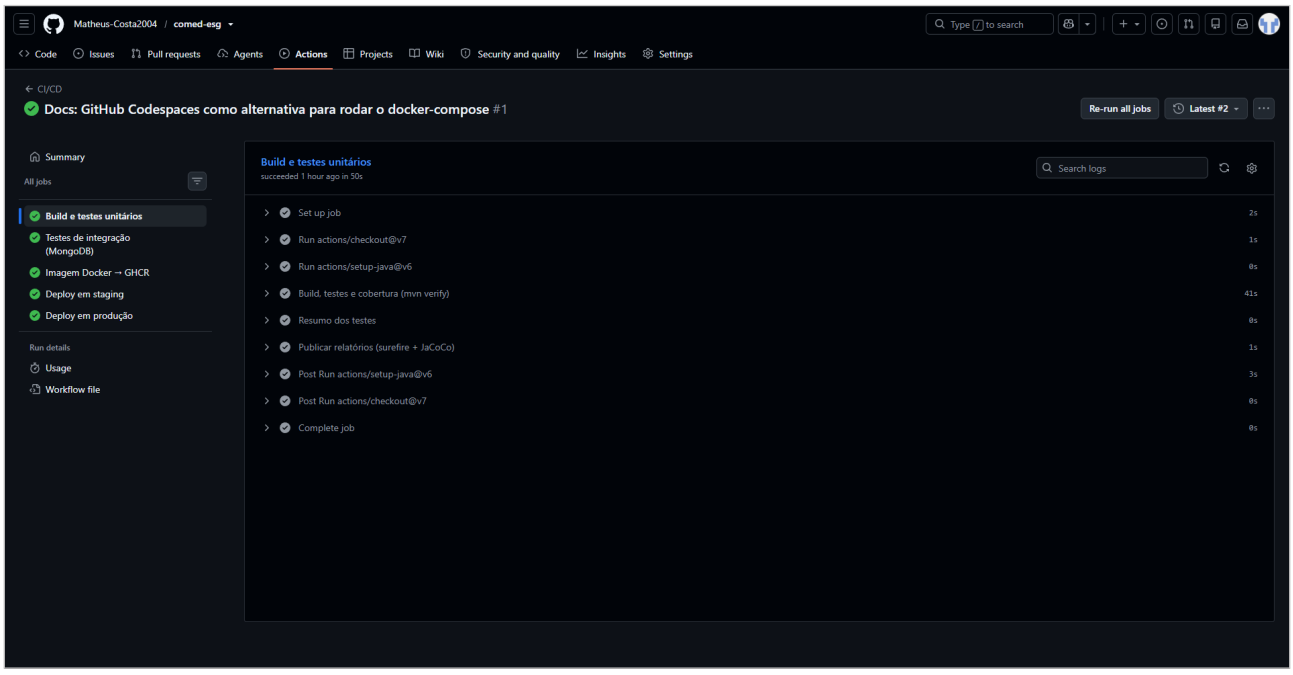


Figura 3 — Job de build e testes unitários com todos os passos concluídos.

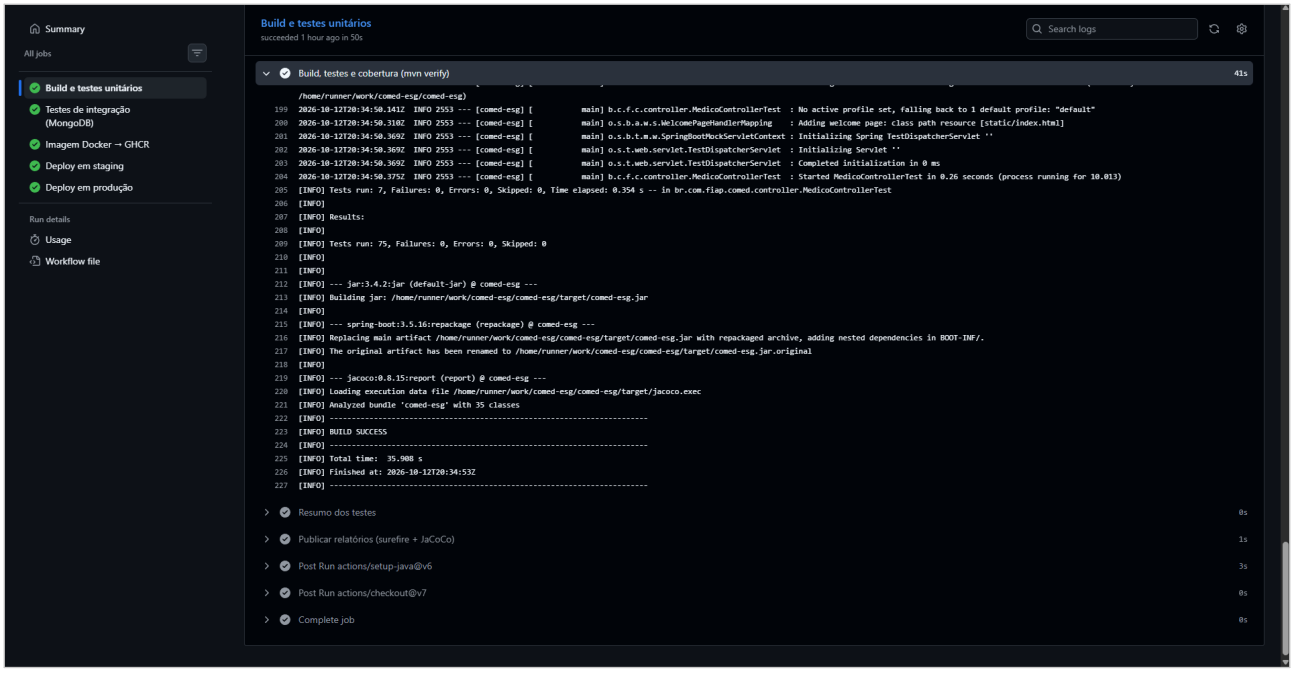


Figura 4 — Log do mvn verify: Tests run: 75, Failures: 0, Errors: 0; relatório JaCoCo e BUILD SUCCESS.

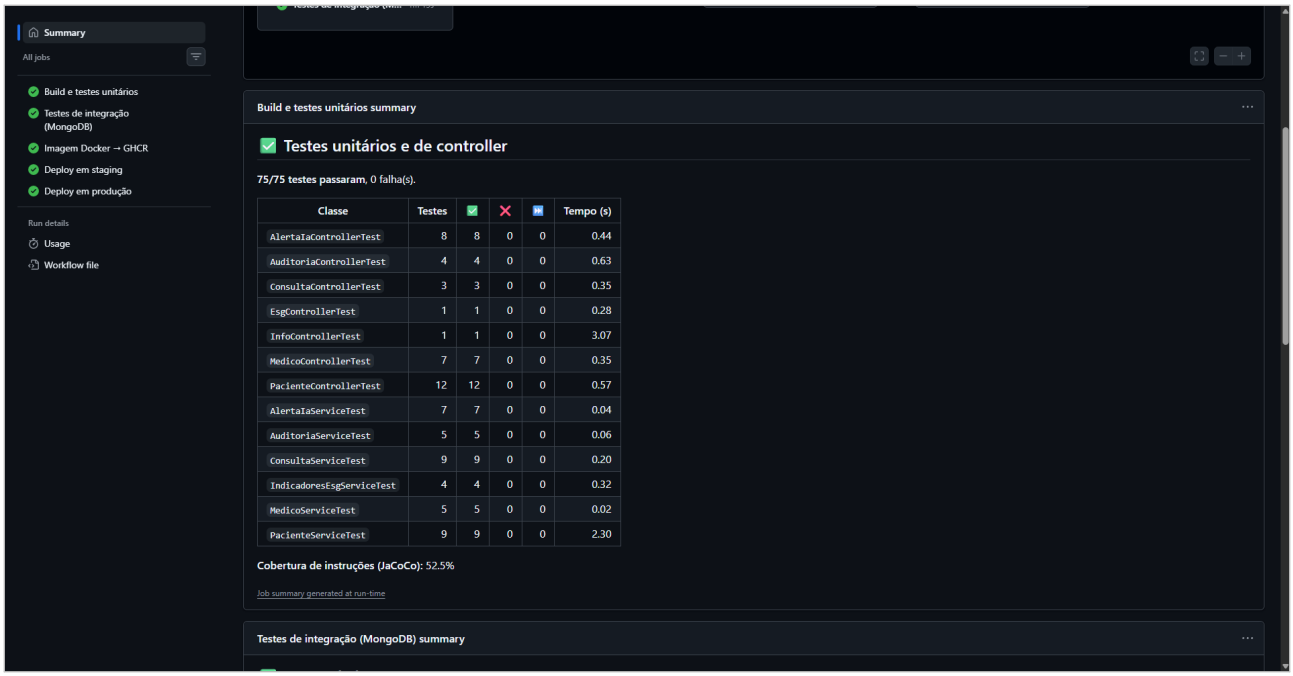


Figura 5 — Resumo gerado pelo pipeline: 75/75 testes por classe e cobertura de instruções de 52,5%.

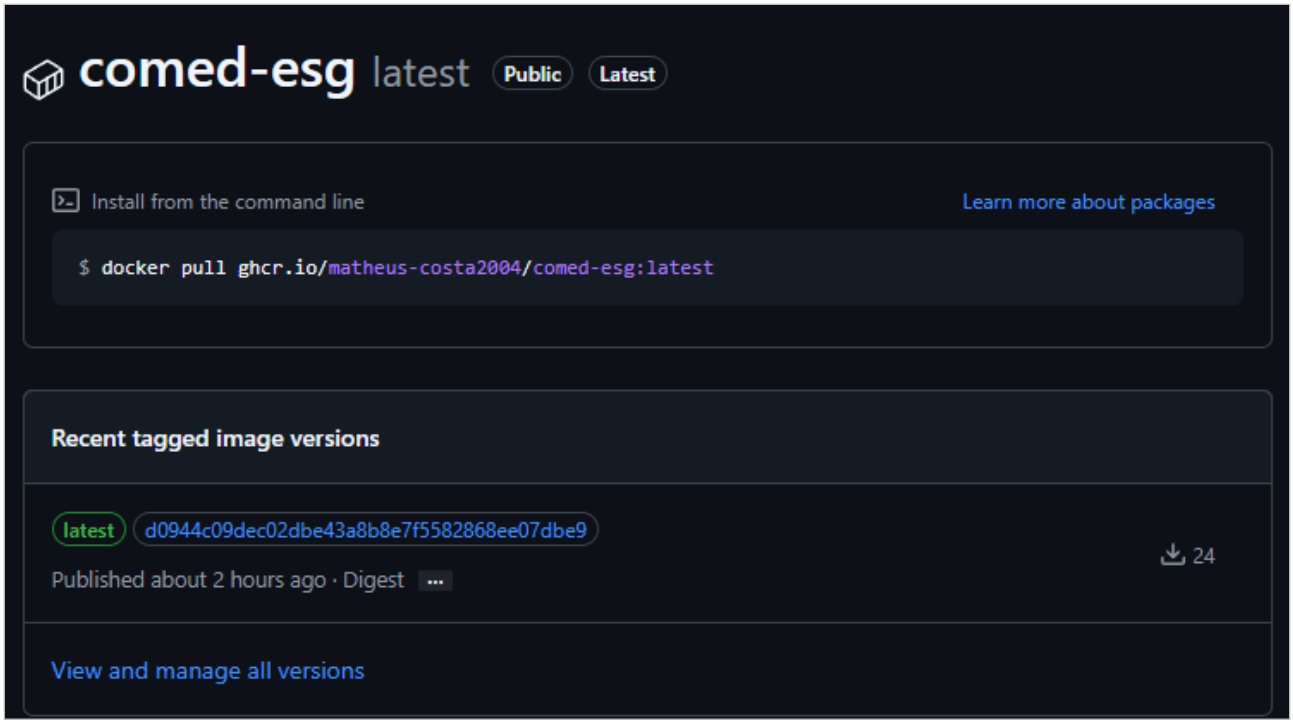


Figura 6 — Imagem publicada no GHCR com as tags latest e d0944c0..., pública e com o comando docker pull.

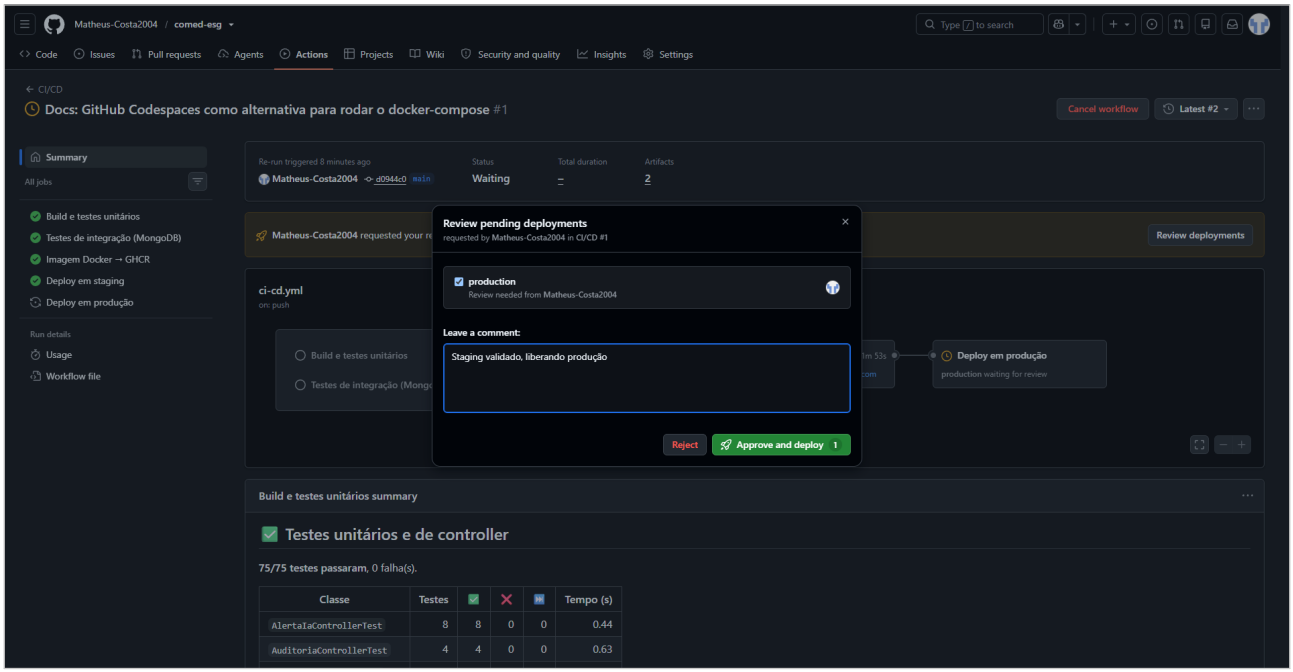


Figura 7 — Gate de produção: o pipeline aguarda revisão e o deploy é aprovado com comentário.

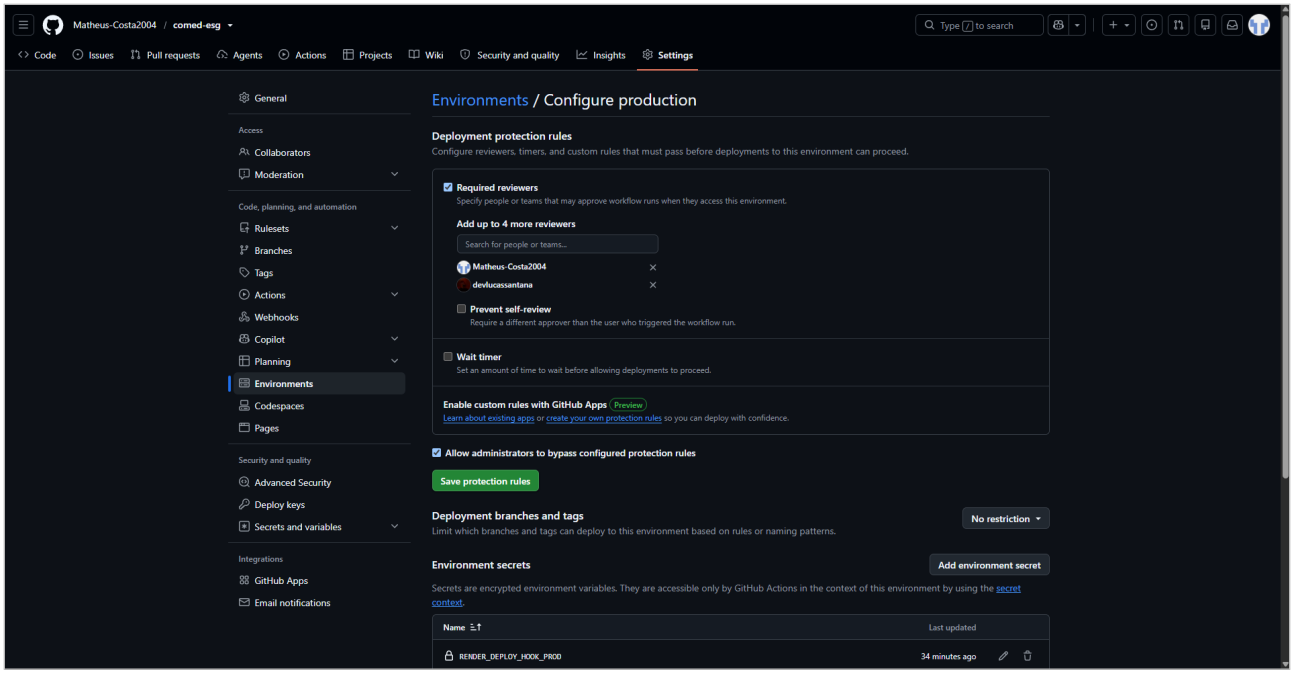


Figura 8 — Environment production com Required reviewers (Matheus-Costa2004 e devlucassantana) e o secret do Deploy Hook.

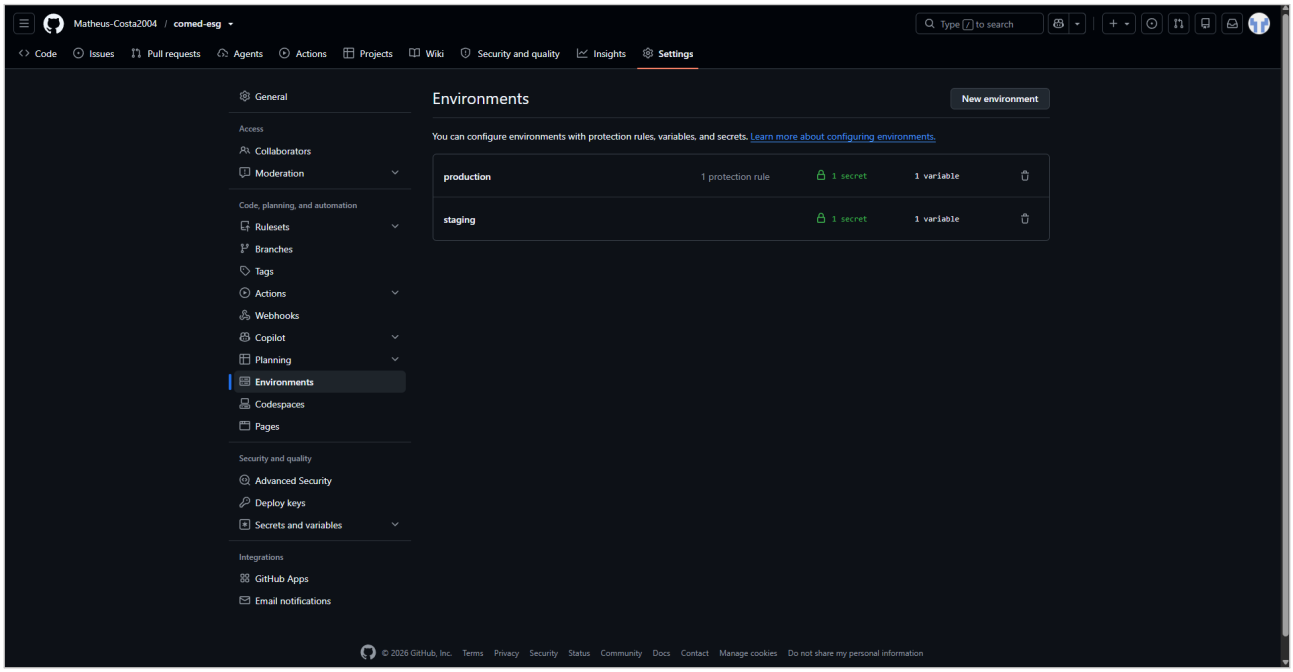


Figura 9 — Environments do repositório: production com regra de proteção; staging sem regra (deploy contínuo).

5. Evidências dos ambientes staging e produção

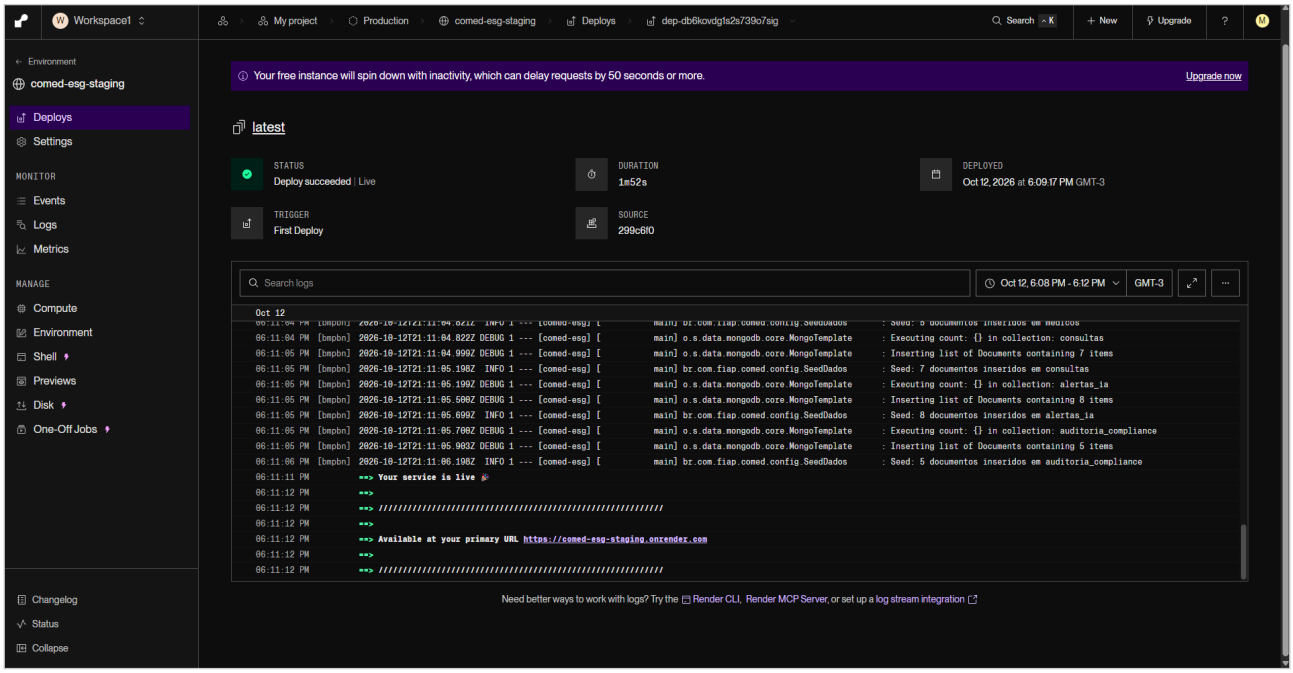


Figura 10 — Render, serviço comed-esg-staging: Deploy succeeded | Live, com o seed inserindo os dados nas 5 collections.

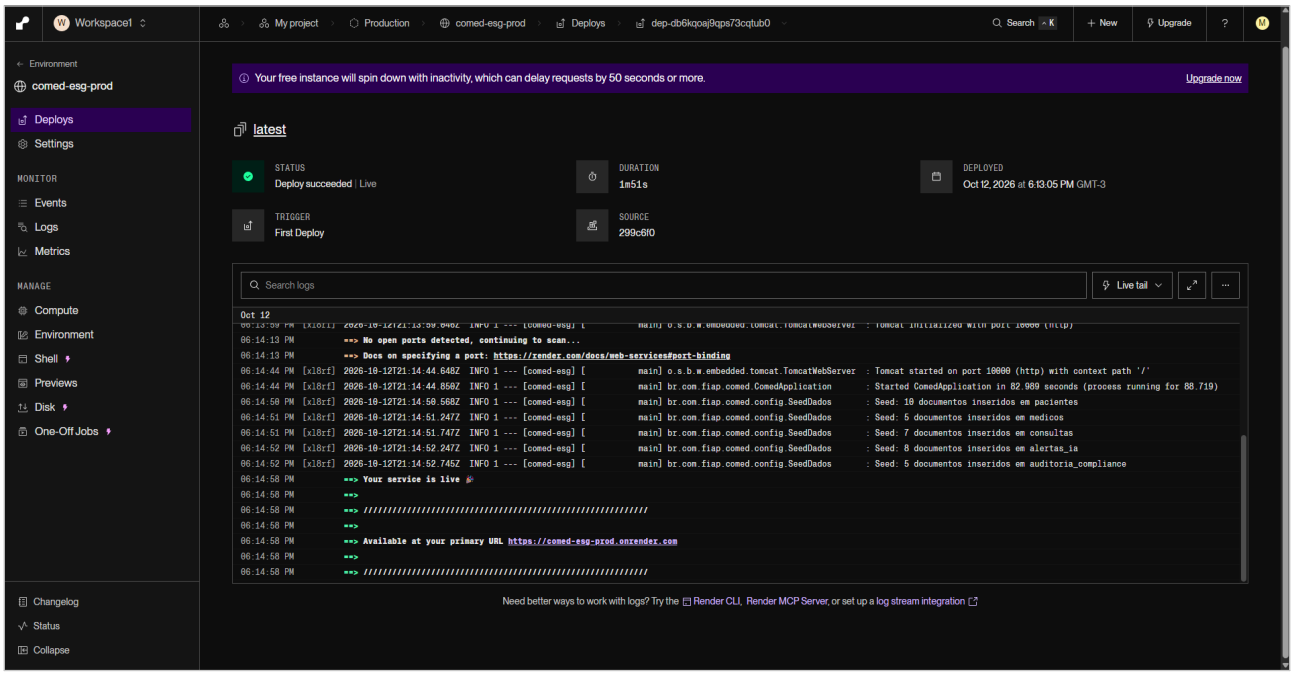


Figura 11 — Render, serviço comed-esg-prod: Deploy succeeded | Live.

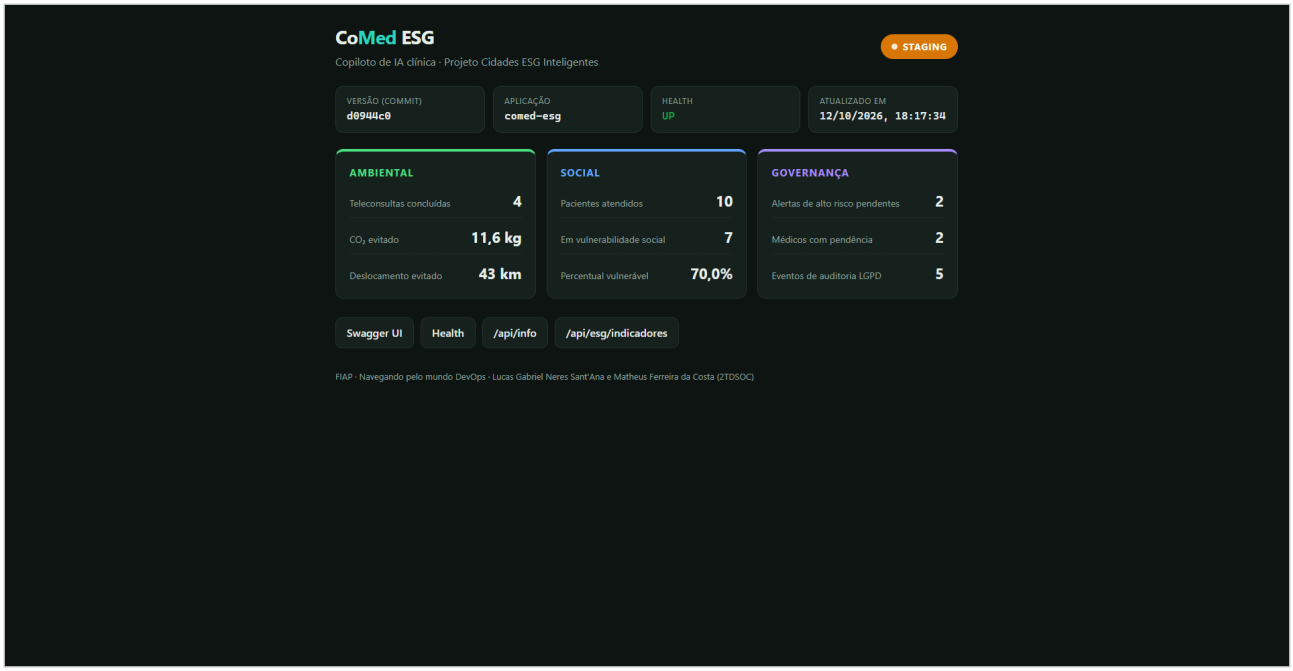


Figura 12 — Staging no ar: selo STAGING, versão d0944c0, health UP e indicadores ESG.

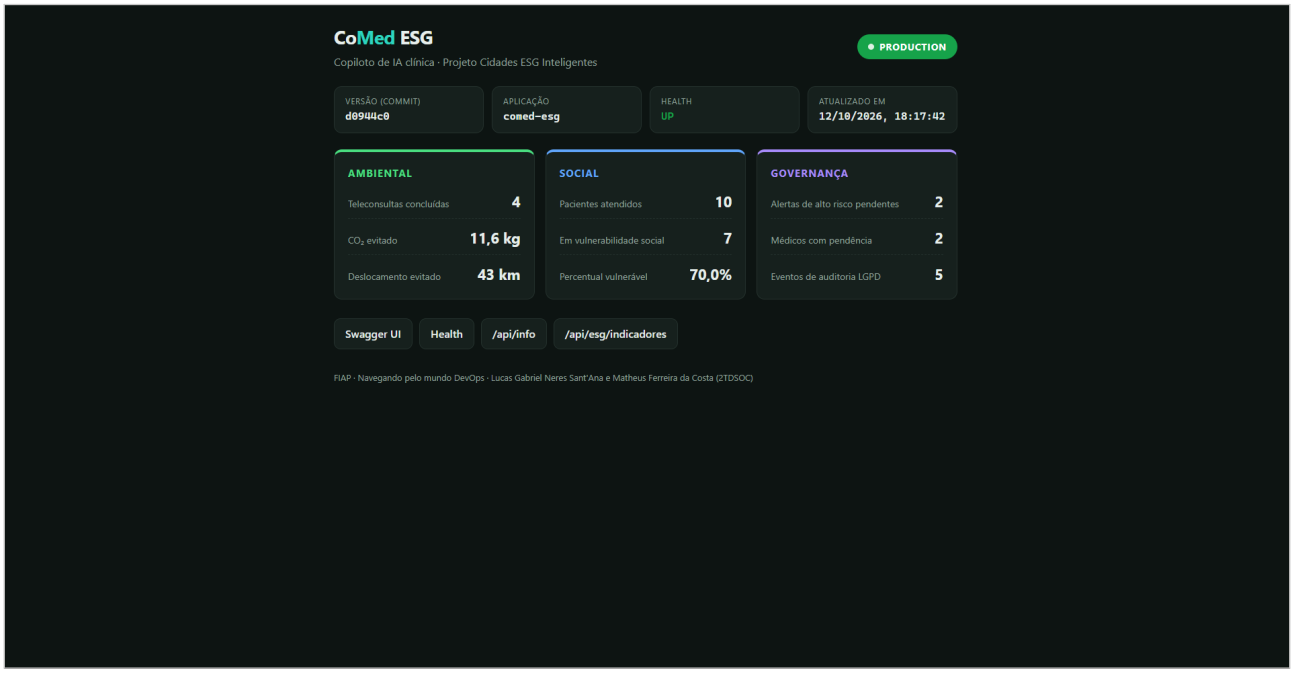


Figura 13 — Produção no ar: selo PRODUCTION, mesma versão d0944c0 promovida de staging, health UP.

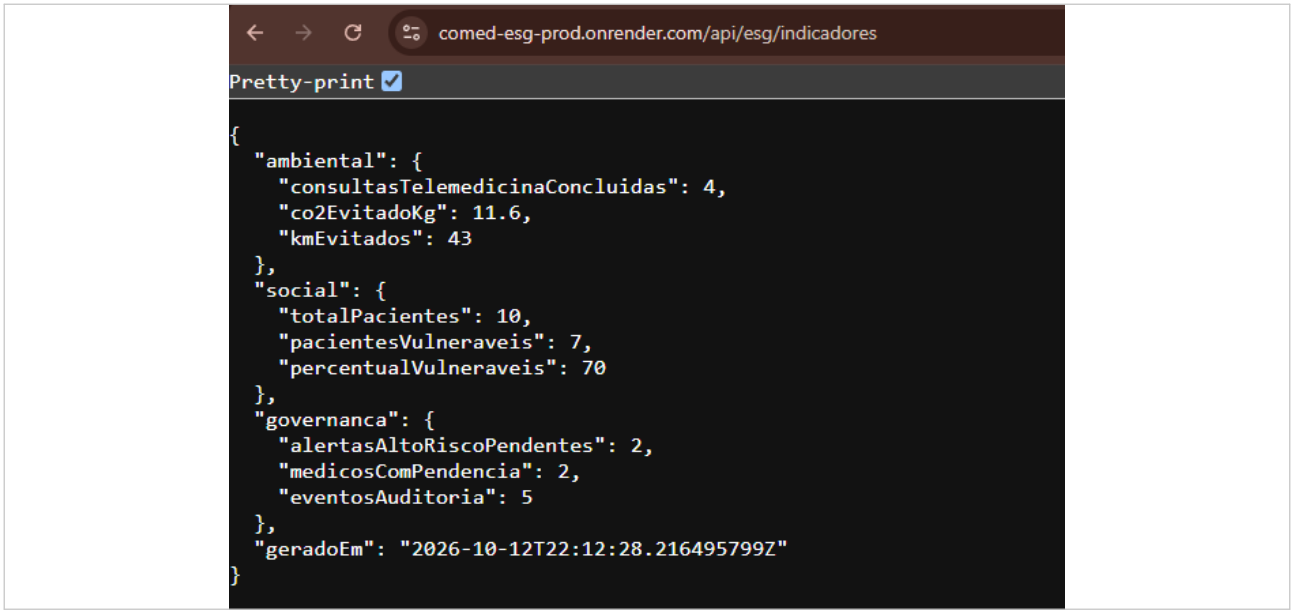


Figura 14 — GET /api/esg/indicadores em produção: 4 teleconsultas, 11,6 kg de CO2 e 43 km evitados; 70% de pacientes em vulnerabilidade; 2 alertas pendentes, 2 médicos com pendência e 5 eventos de auditoria.

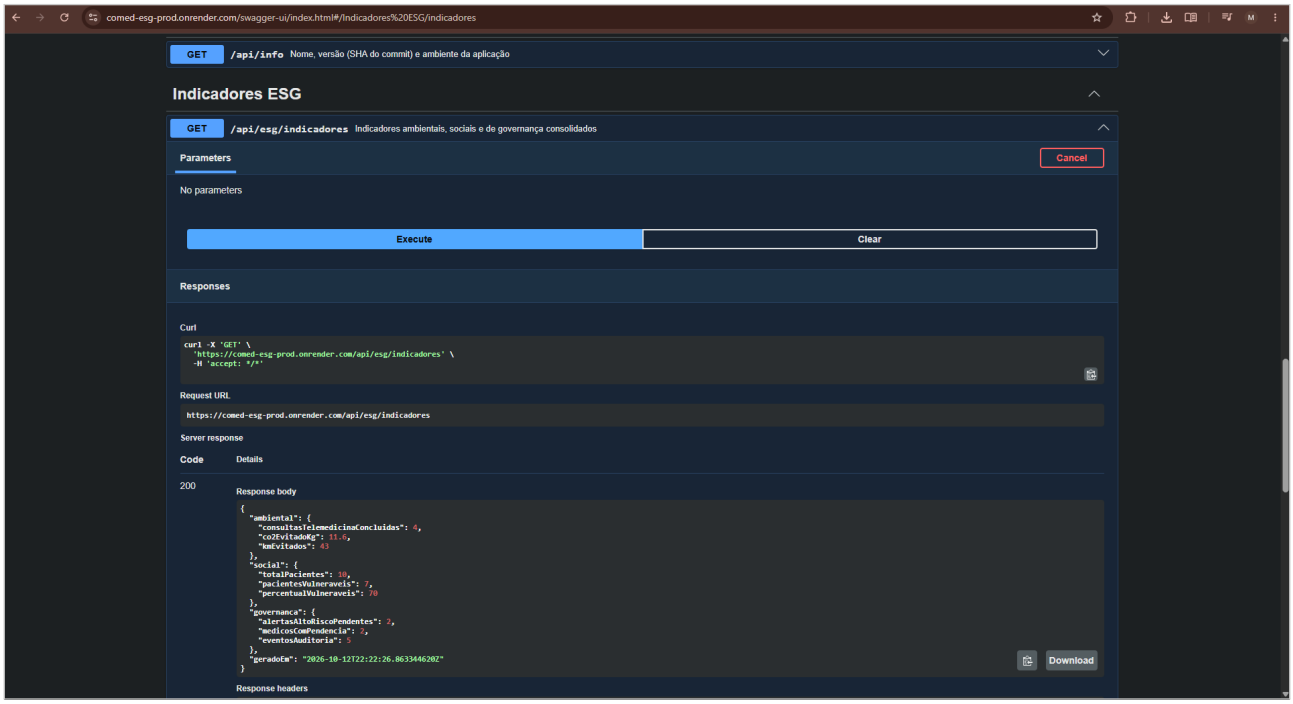


Figura 15 — Swagger UI em produção: chamada a /api/esg/indicadores com resposta 200.

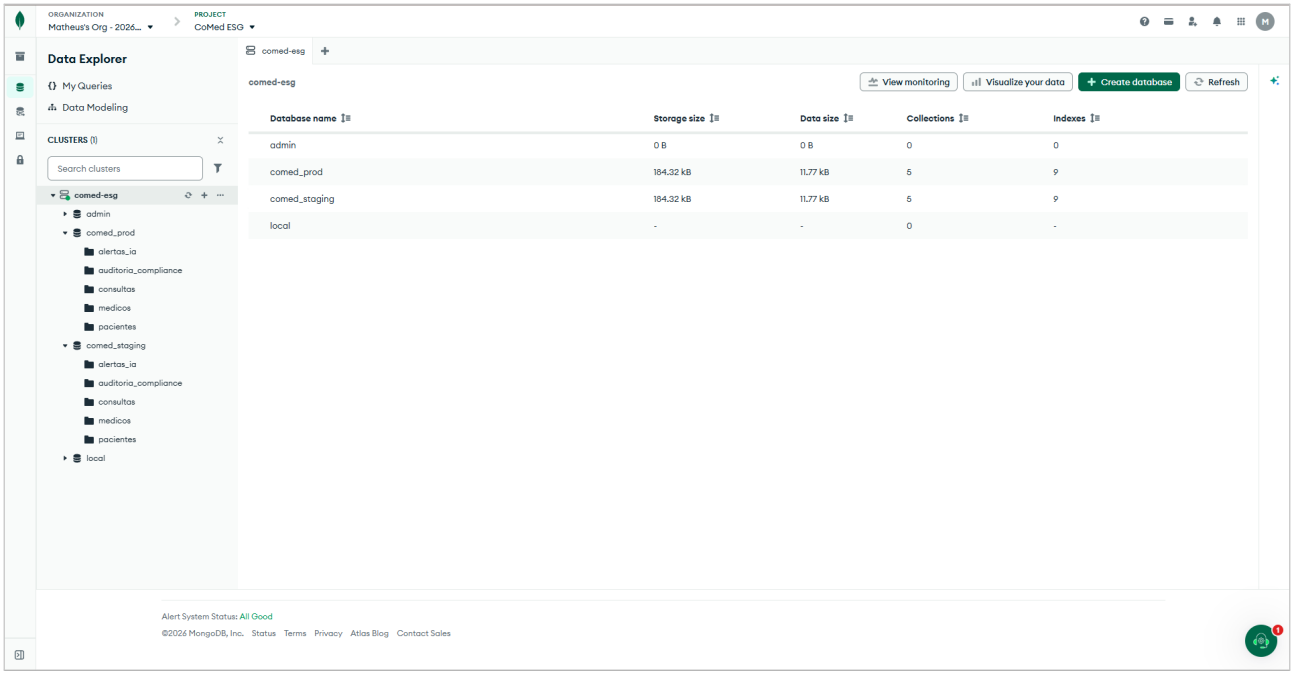


Figura 16 — MongoDB Atlas: databases comed_staging e comed_prod, cada um com as 5 collections do modelo ESG.

Ambiente local com Docker Compose

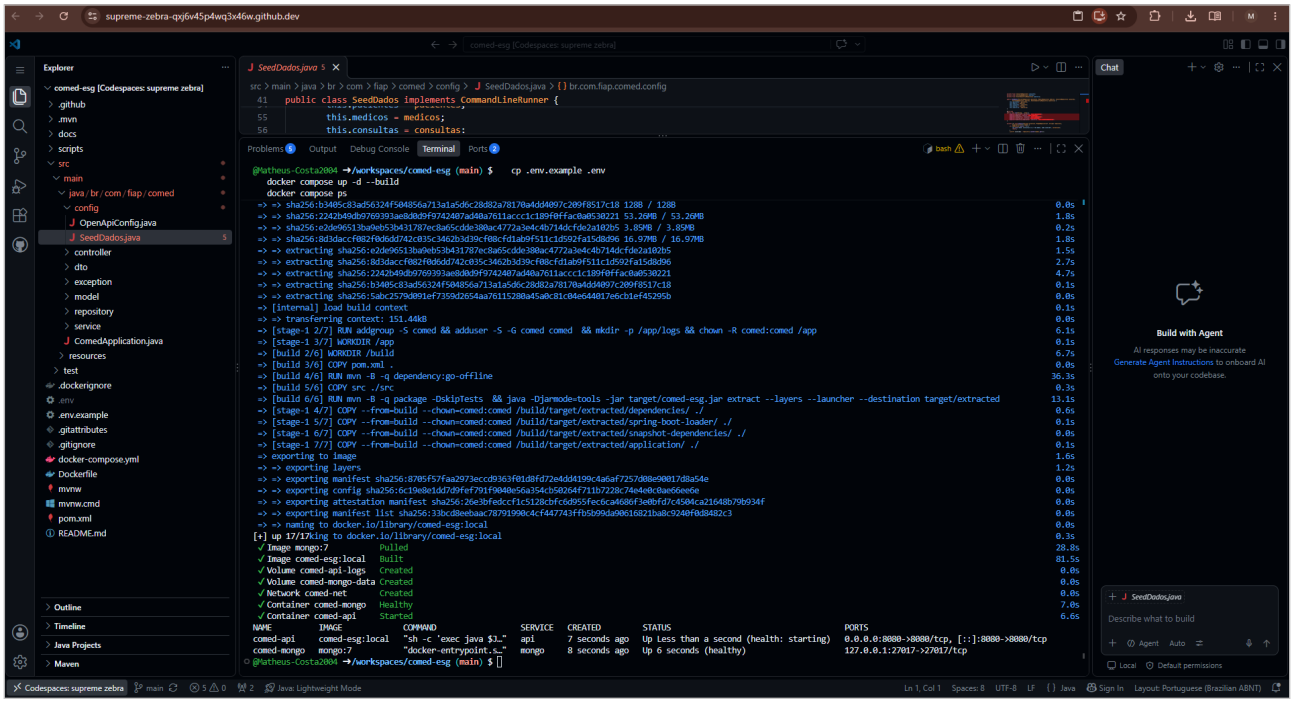


Figura 17 — docker compose up --build no GitHub Codespace: imagem construída, volumes comed-api-logs e comed-mongo-data, rede comed-net e containers criados.

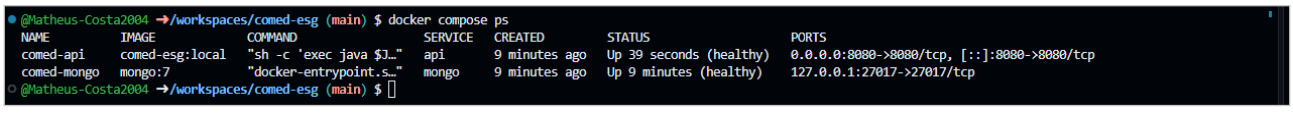


Figura 18 — docker compose ps: comed-api e comed-mongo healthy.

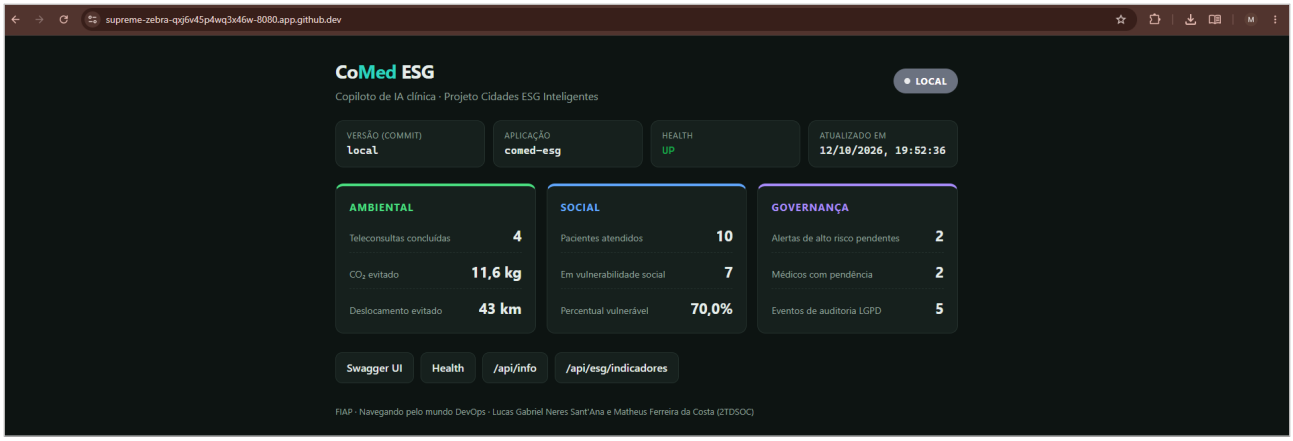


Figura 19 — Aplicação local servida pelo Compose (porta 8080 do Codespace), com o selo LOCAL.

6. Desafios encontrados e como foram resolvidos

Desafio	Problema	Solução
Azure DevOps sem agentes gratuitos	Organizações novas não recebem mais os agentes hospedados automaticamente; a liberação depende de formulário e leva dias.	Pipeline migrado para GitHub Actions, que oferece runners hospedados e Environments com aprovação.
Docker Desktop não iniciava no Windows	O hipervisor não subia (evento 44 do Hyper-V-Hypervisor, “No Execute feature (NX)”), mesmo com virtualização ativa no firmware, Virtual Machine Platform habilitado e hypervisorlaunchtype Auto. Limitação da placa B450M-DA.	Diagnóstico pelos logs de eventos do Windows; o Compose passou a ser executado num GitHub Codespace, documentado no README.
Containers sem comunicação no Codespace	A API não alcançava o Mongo (SocketTimeoutException em mongo:27017), embora os dois estivessem na rede comed-net e o mongod usasse --bind_ip_all. O firewall do Docker interno bloqueava o encaminhamento.	Isolado com docker network inspect e wget de dentro do container; resolvido com iptables-legacy -P FORWARD ACCEPT na VM do Codespace.
Cold start do Render Free	Instâncias gratuitas levam minutos para subir uma nova versão, e o Spring Boot demora com 0,1 CPU.	Smoke test com polling de até 12 min que só aprova quando /api/info devolve o SHA esperado e o health está UP.
Senha do banco no Atlas	O Atlas não exibe a senha de um usuário existente, e a credencial não pode ir para o repositório.	Senha regenerada no Atlas e colocada apenas na variável MONGODB_URI de cada serviço no Render; usuário dedicado comed_app com readWrite só nos dois databases.
Nome da imagem no GHCR	O GHCR exige nomes em minúsculas, e o usuário do GitHub tem maiúsculas (Matheus-Costa2004).	O workflow converte o dono do repositório para minúsculas (${GITHUB_REPOSITORY_OWNER,,}).
Primeira execução sem configuração	Na primeira execução os serviços do Render e os secrets ainda não existiam.	O script de deploy falha com mensagem explicando o secret ausente; após a configuração, o pipeline foi reexecutado com sucesso.
Aprovação manual no plano gratuito	Required reviewers em Environments só está disponível em repositório público no plano gratuito.	Repositório público; staging sem proteção e produção com dois revisores.

7. Tecnologias utilizadas

Área	Tecnologias
Aplicação	Java 21, Spring Boot 3.5.16, Spring Web, Spring Data MongoDB, Bean Validation, Actuator, springdoc-openapi (Swagger UI)
Testes	JUnit 5, Mockito, Spring MockMvc (WebMvcTest), testes de integração com @SpringBootTest, JaCoCo, Maven Surefire e Failsafe
Banco de dados	MongoDB 7 (local, em container) e MongoDB Atlas (staging e produção)
Containers	Docker (multi-stage), Docker Compose, GitHub Container Registry (GHCR)

Área	Tecnologias
CI/CD	GitHub Actions, GitHub Environments (secrets, variables e Required reviewers)
Hospedagem	Render (Web Services a partir de imagem, Deploy Hooks); GitHub Codespaces para o ambiente Compose

8. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	✔
Dockerfile funcional	✔
docker-compose.yml ou arquivos Kubernetes	✔
Pipeline com etapas de build, teste e deploy	✔
README.md com instruções e prints	✔
Documentação técnica com evidências (PDF ou PPT)	✔
Deploy realizado nos ambientes staging e produção	✔

Evidências de cada item: estrutura do repositório e do ZIP (seção 3), Dockerfile e Compose (seção 3 e Figuras 17–19), pipeline (seção 2 e Figuras 2–9), deploy em staging e produção (Figuras 10–16).